

```
!pip install torch
!pip install torch_geometric
```

```
Requirement already satisfied: torch in c:\users\asus\anaconda3\lib\
site-packages (2.7.1+cu118)
Requirement already satisfied: filelock in c:\users\asus\anaconda3\
lib\site-packages (from torch) (3.17.0)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\
asus\anaconda3\lib\site-packages (from torch) (4.12.2)
Requirement already satisfied: sympy>=1.13.3 in c:\users\asus\
anaconda3\lib\site-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in c:\users\asus\anaconda3\
lib\site-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in c:\users\asus\anaconda3\lib\
site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in c:\users\asus\anaconda3\lib\
site-packages (from torch) (2025.3.2)
Requirement already satisfied: setuptools in c:\users\asus\anaconda3\
lib\site-packages (from torch) (72.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\asus\
anaconda3\lib\site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\asus\
anaconda3\lib\site-packages (from jinja2->torch) (3.0.2)
Requirement already satisfied: torch_geometric in c:\users\asus\
anaconda3\lib\site-packages (2.7.0)
Requirement already satisfied: aiohttp in c:\users\asus\anaconda3\lib\
site-packages (from torch_geometric) (3.11.10)
Requirement already satisfied: fsspec in c:\users\asus\anaconda3\lib\
site-packages (from torch_geometric) (2025.3.2)
Requirement already satisfied: jinja2 in c:\users\asus\anaconda3\lib\
site-packages (from torch_geometric) (3.1.6)
Requirement already satisfied: numpy in c:\users\asus\anaconda3\lib\
site-packages (from torch_geometric) (2.1.3)
Requirement already satisfied: psutil>=5.8.0 in c:\users\asus\
anaconda3\lib\site-packages (from torch_geometric) (5.9.0)
Requirement already satisfied: pyparsing in c:\users\asus\anaconda3\
lib\site-packages (from torch_geometric) (3.2.0)
Requirement already satisfied: requests in c:\users\asus\anaconda3\
lib\site-packages (from torch_geometric) (2.32.3)
Requirement already satisfied: tqdm in c:\users\asus\anaconda3\lib\
site-packages (from torch_geometric) (4.67.1)
Requirement already satisfied: xxhash in c:\users\asus\anaconda3\lib\
site-packages (from torch_geometric) (3.6.0)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in c:\users\
asus\anaconda3\lib\site-packages (from aiohttp->torch_geometric)
(2.4.4)
Requirement already satisfied: aiosignal>=1.1.2 in c:\users\asus\
anaconda3\lib\site-packages (from aiohttp->torch_geometric) (1.2.0)
Requirement already satisfied: attrs>=17.3.0 in c:\users\asus\
anaconda3\lib\site-packages (from aiohttp->torch_geometric) (24.3.0)
```

```
Requirement already satisfied: frozenlist>=1.1.1 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp->torch_geometric) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp->torch_geometric) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp->torch_geometric) (0.3.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp->torch_geometric) (1.18.0)
Requirement already satisfied: idna>=2.0 in c:\users\asus\anaconda3\lib\site-packages (from yarll<2.0,>=1.17.0->aiohttp->torch_geometric) (3.7)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\asus\anaconda3\lib\site-packages (from jinja2->torch_geometric) (3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\asus\anaconda3\lib\site-packages (from requests->torch_geometric) (3.3.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\asus\anaconda3\lib\site-packages (from requests->torch_geometric) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\asus\anaconda3\lib\site-packages (from requests->torch_geometric) (2026.1.4)
Requirement already satisfied: colorama in c:\users\asus\anaconda3\lib\site-packages (from tqdm->torch_geometric) (0.4.6)
```

Load Dataset

```
import pandas as pd

df = pd.read_csv("data/elite_data/enriched.csv")
df = df.sample(100000, random_state=42)

print(df.shape)

(100000, 30)
```

Create Node ID Mappings

```
accounts = pd.Index(pd.concat([df["sender_account"],
df["receiver_account"]]).unique())
devices = pd.Index(df["device_hash"].unique())
ips = pd.Index(df["ip_address"].unique())
locations = pd.Index(df["location"].unique())
merchants = pd.Index(df["merchant_category"].unique())

account_map = {v: i for i, v in enumerate(accounts)}
device_map = {v: i for i, v in enumerate(devices)}
ip_map = {v: i for i, v in enumerate(ips)}
location_map = {v: i for i, v in enumerate(locations)}
merchant_map = {v: i for i, v in enumerate(merchants)}
```

```
print("Accounts:", len(accounts))
print("Devices:", len(devices))
print("IPs:", len(ips))
print("Locations:", len(locations))
print("Merchants:", len(merchants))
```

```
Accounts: 179590
Devices: 99473
IPs: 99999
Locations: 8
Merchants: 8
```

Build Transaction Edge Index

Transaction edges (Account → Account)

```
import torch
import numpy as np
src = df["sender_account"].map(account_map).values
dst = df["receiver_account"].map(account_map).values

transaction_edge_index = torch.tensor(np.vstack((src, dst)),
dtype=torch.long)

fraud_labels = torch.tensor(df["is_fraud"].astype(int).values)
```

Device edges (Account → Device)

```
src = df["sender_account"].map(account_map).values
dst = df["device_hash"].map(device_map).values

device_edge_index = torch.tensor(np.vstack((src, dst)),
dtype=torch.long)
```

IP edges (Account → IP)

```
src = df["sender_account"].map(account_map).values
dst = df["ip_address"].map(ip_map).values

ip_edge_index = torch.tensor(np.vstack((src, dst)), dtype=torch.long)
```

Location edges (Account → Location)

```
src = df["sender_account"].map(account_map).values
dst = df["location"].map(location_map).values

location_edge_index = torch.tensor(np.vstack((src, dst)),
dtype=torch.long)
```

Merchant edges (Account → Merchant)

```
src = df["sender_account"].map(account_map).values
dst = df["merchant_category"].map(merchant_map).values

merchant_edge_index = torch.tensor(np.vstack((src, dst)),
dtype=torch.long)
```

Account Node Features

```
from sklearn.preprocessing import StandardScaler
account_features = df.groupby("sender_account")[[
    "velocity_score",
    "spending_deviation_score",
    "geo_anomaly_score",
    "tx_count_24h",
    "amt_sum_24h",
    "device_fraud_rate",
    "ip_fraud_rate"
]].mean()

device_features = df.groupby("device_hash")
[["device_fraud_rate"]].mean()
device_features=device_features.reindex(devices).fillna(0)
device_x=torch.tensor(device_features.values,dtype=torch.float)

ip_features = df.groupby("ip_address")[[["ip_fraud_rate"]].mean()
ip_features=ip_features.reindex(ips).fillna(0)
ip_x=torch.tensor(ip_features.values,dtype=torch.float)

scaler=StandardScaler()
account_features=pd.DataFrame(scaler.fit_transform(account_features),i
ndex=account_features.index,columns=account_features.columns)
account_features = account_features.reindex(accounts).fillna(0)

account_x = torch.tensor(account_features.values, dtype=torch.float)
```

Build HeteroData Graph

```
from torch_geometric.data import HeteroData

data = HeteroData()

# node features
data["account"].x = account_x
data["account"].num_nodes = len(accounts)

data["device"].x = device_x
data["device"].num_nodes=len(devices)
data["ip"].x=ip_x
data["ip"].num_nodes=len(ips)
```

```

data["location"].num_nodes = len(locations)
data["merchant"].num_nodes = len(merchants)

# edges
data["account", "transaction", "account"].edge_index =
transaction_edge_index
data["account", "transaction", "account"].edge_label = fraud_labels

# ADD THIS
data["account", "transaction", "account"].edge_attr = torch.tensor(
    df["amount_wins"].values,
    dtype=torch.float
).view(-1,1)

data["account", "uses_device", "device"].edge_index = device_edge_index
data["account", "uses_ip", "ip"].edge_index = ip_edge_index
data["account", "located_at", "location"].edge_index =
location_edge_index
data["account", "purchases", "merchant"].edge_index =
merchant_edge_index

```

Save Graph

```

torch.save(data, "data/fraud_graph.pt")

print(data)

HeteroData(
  account={
    x=[179590, 7],
    num_nodes=179590,
  },
  device={
    x=[99473, 1],
    num_nodes=99473,
  },
  ip={
    x=[99999, 1],
    num_nodes=99999,
  },
  location={ num_nodes=8 },
  merchant={ num_nodes=8 },
  (account, transaction, account)={
    edge_index=[2, 100000],
    edge_label=[100000],
    edge_attr=[100000, 1],
  },
  (account, uses_device, device)={ edge_index=[2, 100000] },
  (account, uses_ip, ip)={ edge_index=[2, 100000] },
  (account, located_at, location)={ edge_index=[2, 100000] },

```

```
(account, purchases, merchant)={ edge_index=[2, 100000] }  
)
```